Table des matières

Description des flux de données - moyun-pos SCE 2.0 1
1. Début de service - Pointage entrée (Événement S) . . . 1
2. Ouverture d'une table - Commande (Événement P) . . . 1
3. Le client paie - Vente (Événement N) . . . 2
4. Rapport X en cours de service . . . 2
5. Fin de journée - Rapport Z (Événement R, double enveloppe) . . . 2
6. Fin de service - Pointage sortie (Événement S) . . . 3
7. Exception - Facture B2B (Événement I + Peppol) . . . 3
8. Exception - FDM injoignable en cours de service . . . 3
9. Récapitulatif de persistance . . . 3

Description des flux de données - moyun-pos SCE 2.0

Ce document parcourt une journée de service complète chez Tokyo Sushi Bar à travers le système moyun-pos, en montrant quelles données circulent et où à chaque étape. Il complète `DOCUMENT_PROCESSUS_TECHNIQUE_FR.md` (qui décrit l'architecture statique) en suivant les données le long d'un parcours utilisateur.

1. Début de service - Pointage entrée (Événement S)

Action utilisateur. L'opérateur démarre le kiosque. Le compositeur Cage Wayland lance `moyun-pos` en plein écran. L'écran de login demande un INSZ. L'opérateur entre 11 chiffres sur le pavé numérique.

Circulation des données.

1. UI (`LoginView.vue`) → IPC `commands::auth::login(insz)`.
2. `pos-db::queries::users::find_by_insz` → ligne SQLite `users`.
3. Le store Pinia auth enregistre `(insz, name, role)`.
4. L'UI redirige vers `TablesView.vue`. Le store appelle automatiquement `commands::social::work_in()` - un événement fiscal S :
   - Clé d'idempotence construite à partir de `pos_id`, `pos_date_time`, `terminal_id`, `"WORK_IN"`, et du prochain compteur séquentiel.
   - Corps JSON canonique POST sur l'endpoint GraphQL du FDM.
   - Sur succès : le FDM renvoie short signature + compteurs ; ligne ajoutée à `fiscal_events` en une seule transaction ; ligne `social_status` dans `users` basculée à `present`.
   - Le ticket n'est pas imprimé pour les événements S.

2. Ouverture d'une table - Commande (Événement P)

Action utilisateur. L'opérateur tape Table 12 ; la grille du menu se charge. Les clients commandent trois plats ; l'opérateur appuie sur "Envoyer en cuisine".

Circulation des données.

1. UI (`OrderView.vue`) appelle `commands::order::add_order_line` une fois par plat. Chaque appel écrit une ligne `order_lines` avec le nom dénormalisé, le label TVA, le prix unitaire (depuis `menu_items`), et les éventuels modificateurs.
2. "Envoyer en cuisine" déclenche `commands::tables::sign_table_order` :
   - Construit un `OrderInput` pour P (pro forma) - même forme qu'une vente mais sans paiement.
   - Idempotency-keyed et sérialisé en JSON canonique.
   - Le FDM signe ; on archive l'événement P.
   - `pos-print::receipt::kitchen` construit un payload ESC/POS et l'envoie à l'imprimante cuisine (destination TCP distincte de l'imprimante du ticket client principal).

Le compteur fiscal P avance de 1. Le compteur N reste intact jusqu'à ce que le client paie effectivement.

3. Le client paie - Vente (Événement N)

Action utilisateur. Le client demande l'addition. L'opérateur tape "Payer" sur la table 12. Choisit "Carte" . Le terminal carte renvoie OK. L'opérateur confirme.

Circulation des données.

1. UI (`CheckoutView.vue`) → `commands::checkout::checkout` avec `{order_id, payment_method: "card", amount_cents}`.
2. Vérification du statut FDM. `FdmClient::status` s'exécute en premier. Si non-opérationnel, la commande retourne `FdmUnreachable` et l'UI affiche une modale bloquante. Aucun compteur n'est consommé, aucune ligne DB n'est écrite.
3. Construit `SaleInput` à partir des lignes de commande. Ajoute la référence FDM vers l'événement P antérieur de la table 12 pour que le FDM les relie.
4. Sérialisation JSON canonique et appel `FdmClient::sign_sale`.
5. Sur succès FDM, à l'intérieur d'une seule transaction `BEGIN IMMEDIATE` :
   — Allocation du prochain compteur N (`fiscal_counters`).
   — Ajout de l'événement fiscal à `fiscal_events`.
   — Marquage de la ligne `orders` `status='closed'`.
   — Écriture de la ligne `payments`.
6. `pos-print::receipt::sale` construit le ticket client :
   — En-tête (nom du restaurant, adresse, TVA).
   — Détail ligne par ligne.
   — Récapitulatif TVA (A=21%, B=12%, C=6%).
   — Bloc de données de contrôle issu de la réponse FDM.
   — QR Code 25×25 (Type 2) à partir de l'URL 38 caractères fournie par le FDM.
7. Trame ESC/POS envoyée par TCP au TM-m30III. Le tiroir-caisse n'est PAS ouvert (paiement carte).

4. Rapport X en cours de service

Action utilisateur. Le gérant tape "Rapport X" dans la vue rapports pour voir les totaux en cours.

Circulation des données.

1. UI → `commands::report::generate_x_report`.
2. `pos-db::queries::reports::get_turnover_summary_for_date` retourne les agrégats de la période de réservation ouverte.
3. Construit `ReportTurnoverXInput` (pas de champ booking-date dans les rapports X selon v1.3).
4. Le FDM signe un événement R labellisé `R-X-TURNOVER`. Archivé dans `fiscal_events`.
5. Ticket imprimé pour le gérant. Les compteurs ne sont pas remis à zéro.

5. Fin de journée - Rapport Z (Événement R, double enveloppe)

Action utilisateur. Le gérant tape "Clôture journée" .

Circulation des données.

1. UI → `commands::report::generate_z_report`.
2. La commande émet deux événements R de manière atomique :
   — `R-Z-TURNOVER` (`ReportTurnoverZInput`, inclut `report_booking_date`).
   — `R-Z-USER` (`ReportUserZInput`).
3. À l'intérieur d'une seule transaction SQLite :
   — Allocation de deux compteurs R consécutifs.
   — POST des deux inputs au FDM (séquentiellement, idempotency-keyed).
   — Sur succès : ajout des deux lignes `fiscal_events` ; insertion d'une ligne `z_reports` référençant l'événement turnover.
   — Si l'un ou l'autre appel FDM échoue, toute la tx est rollback - aucun état partiel n'est persisté.
4. Sauvegarde de la DB SQLite vers `/var/lib/moyun-pos/backups/YYYY-MM-DD.db` via l'API Online Backup.

5. Deux tickets de rapport Z sont imprimés.

6. Fin de service - Pointage sortie (Événement S)

Action utilisateur. L'opérateur tape "Pointer sortie".

Circulation des données.

1. UI → `commands::social::work_out`.
2. Événement S construit (flag `work_out=true`). Idempotence, JSON canonique, FDM, archivage. `users.social_status` basculé à `absent`.
3. L'UI déconnecte, retour à l'écran de login.

7. Exception - Facture B2B (Événement I + Peppol)

Action utilisateur. Pendant le checkout, l'opérateur active "Générer facture B2B", saisit le n° TVA, le nom, l'adresse et le pays de l'acheteur.

Circulation des données.

1. Le checkout standard se déroule (Événement N si vente à emporter, sinon en ligne dans le flux dine-in P/N).
2. Additionnellement, `commands::checkout` invoque `sign_invoice` pour émettre un événement I référençant la vente sous-jacente (via `fdmRefs`).
3. Après l'archivage de l'événement fiscal, `commands::invoice::issue_b2b_invoice` est appelé avec les détails du fournisseur + de l'acheteur et les lignes de vente :
   — `pos-peppol::ubl::to_ubl_xml` construit un UBL BIS Billing 3.0.
   — `AccessPoint::submit` envoie l'UBL à l'AP configuré (actuellement `LoggingAccessPoint` ; câblage Storecove suivi dans `DECISION_PEPPOL_AP_FR.md`).
   — Ligne ajoutée à `peppol_deliveries` avec statut `accepted` / `failed` et l'UBL complet conservé pour audit.
4. Le ticket imprimé montre le n° TVA de l'acheteur et la phrase "Une facture a été émise et délivrée via Peppol".

8. Exception - FDM injoignable en cours de service

Action utilisateur. Une coupure réseau brève survient entre deux commandes.

Circulation des données.

1. Le sondage de fond `FdmClient::status` attrape la déconnexion en premier ; le store Pinia FDM bascule à `unhealthy` ; la pilule `FdmStatus` dans le bandeau passe au rouge.
2. Toute tentative active de "Payer" échoue à la vérification préalable (`FdmUnreachable`) ; l'opérateur voit la modale bloquante.
3. Les commandes pro forma (Événement P) et les ouvertures de tiroir (F-WAR) sont mises en attente dans `fiscal_events` avec `fdm_response_json` à null et le statut `pending`. Elles réémettent à la reconnexion, dans l'ordre des compteurs.
4. Une fois le FDM à nouveau opérationnel, la modale se referme automatiquement et l'opérateur retente le paiement avec le même `pos_fiscal_ticket_no` - le FDM le traite comme un retry idempotent, renvoyant la même signature qu'il aurait renvoyée avant.

9. Récapitulatif de persistance

Action	Tables touchées
Login	`users` (lecture)
Pointage entrée/sortie	`users` (écriture), `fiscal_events` (append S)
Ajout ligne commande	`orders` (insert/update), `order_lines` (insert)
Envoi cuisine	`fiscal_events` (append P)

Action	Tables touchées
Paiement	`fiscal_events` (append N), `payments` (insert), `orders` (close), `fiscal_counters` (advance N)
Rapport X	`fiscal_events` (append R-X)
Rapport Z	`fiscal_events` (append R-Z-TURNOVER + R-Z-USER), `z_reports` (insert), fichier de backup DB
Facture B2B	`fiscal_events` (append I), `peppol_deliveries` (insert)
Remboursement	`fiscal_events` (append N M111/M112 référençant l'original)
Ouverture tiroir (F-WAR)	`fiscal_events` (append F)
Mouvement caisse	`fiscal_events` (append F), `payments` (écriture)

Toute écriture fiscale est encadrée par une seule transaction `BEGIN IMMEDIATE` qui englobe l'allocation du compteur, l'archivage de l'aller-retour FDM, et tout effet de bord sur la commande. Soit tout commit, soit tout rollback.